UNIVALI — Sistemas Distribuídos — Trabalho 2

David Smith
Samuel Cesário Pereira Trizotto de Andrade

REPLICAÇÃO DE ESTADO E CONFIGURAÇÃO COM CONSENSO

etcd (Raft) e Apache ZooKeeper (ZAB): experimentos de liderança, quórum, partição e recuperação

Professor: Ramices dos Santos Silva
28/10/2026

SUMÁRIO

1. Identificação

2. Resumo

3. Fundamentação

4. Ambiente e reprodutibilidade

5. Experimentos

6. Questões comuns (Q1–Q14)

7. Questões específicas do Tema 5 (S1–S4)

8. Quadro-síntese

9. Limitações e ameaças à validade

10. Papel de cada integrante

11. Declaração de uso de ferramentas de IA generativa

12. Referências (ABNT)

1. IDENTIFICAÇÃO

Item	Valor
Instituição / disciplina	UNIVALI — Sistemas Distribuídos — Trabalho 2
Professor	Ramices dos Santos Silva
Integrantes	David Smith; Samuel Cesário Pereira Trizotto de Andrade
Tema	T5 — Replicação de estado e configuração (etcd)
Ênfase	D — ZooKeeper (ZAB)
Repositório	https://github.com/DavidSmithSouzak10/sd-t2-etcd-zookeeper (privado)
Data	28/10/2026

2. RESUMO

Este trabalho investigou a replicação de estado e configuração com o etcd (consenso Raft) e, como ênfase, o Apache ZooKeeper (difusão atômica ZAB). Montamos dois laboratórios só com contêineres Docker: um cluster etcd 3.5.17 com 3 membros e um ensemble ZooKeeper 3.9.6 com 3 servidores, cada um com uma rede interna de consenso e uma rede de clientes, o que permite simular partições sem perder o acesso ao nó isolado. Executamos a bateria comum E1–E8 no etcd e repetimos E3–E8 no ZooKeeper (E9), registrando a previsão da dupla antes de cada experimento e medindo cada tempo pelo menos 5 vezes. Os resultados confirmam o modelo teórico: um único líder por termo, escrita confirmada pela maioria, leitura linearizável por padrão no etcd e local por padrão no ZooKeeper. A queda abrupta do líder levou a uma nova eleição em 1,46 ± 0,18 s no etcd e em 0,38 ± 0,09 s no ZooKeeper; sob partição, o lado minoritário dos dois sistemas parou de aceitar escritas, ou seja, ambos priorizam a consistência. Um membro atrasado recuperou-se pelo log ou, com log compactado, por snapshot, sempre com estado final idêntico. Também medimos efeitos não previstos: a parada ordenada do líder no etcd troca a liderança em cerca de 9 ms, e o `etcdctl elect` usa sessão de 60 s, contra 10 s do `lock`.

3. FUNDAMENTAÇÃO

3.1 O problema: coordenação e máquina de estados replicada

Sistemas distribuídos precisam de um lugar confiável para guardar pouca informação crítica: configuração, quem é o líder de um serviço, quem detém uma

trava, quais nós estão vivos. Guardar isso num único servidor cria um ponto único de falha; copiar em vários servidores sem coordenação cria versões divergentes.

A solução usada pelo etcd e pelo ZooKeeper é a **máquina de estados replicada**: cada réplica começa no mesmo estado e aplica a mesma sequência de comandos, na mesma ordem. Se a sequência é igual, o estado é igual. O problema se reduz a fazer as réplicas concordarem sobre essa sequência mesmo com falhas, isto é, a um problema de **consenso**. O etcd resolve com o Raft e o ZooKeeper com o ZAB. Os dois toleram falhas por parada (*crash*) de uma minoria das réplicas, mas não falhas bizantinas.

3.2 Raft (artigo seminal: ONGARO; OUSTERHOUT, 2014)

O Raft divide o consenso em três partes: - **Eleição de líder.** Cada nó é seguidor, candidato ou líder. O tempo é dividido em **termos** numerados; cada termo tem no máximo um líder. O líder envia *heartbeats* periódicos; um seguidor que fica um *election timeout* sem ouvi-lo vira candidato, incrementa o termo e pede votos. Cada nó vota uma vez por termo, e vence quem obtém a maioria. O timeout é **sorteado** dentro de um intervalo para que raramente dois nós se candidatem ao mesmo tempo (o que dividiria os votos). - **Replicação do log.** Só o líder anexa entradas ao log. Ele as envia aos seguidores (AppendEntries) e considera uma entrada **confirmada** (*committed*) quando a maioria a gravou. Entradas confirmadas são aplicadas à máquina de estados em ordem. - **Segurança.** Um candidato só recebe o voto de quem tem log igual ou mais antigo que o dele (**restrição de log**). Assim, todo líder eleito já contém todas as entradas confirmadas, e nenhuma entrada confirmada é perdida.

Duas extensões ligadas por padrão no etcd 3.5 aparecem nos experimentos: o **pre-vote**, em que um nó só incrementa o termo se uma maioria indicar que votaria nele (evita que um nó isolado force eleições ao voltar), e o **check quorum**, em que o líder deixa o cargo se não ouvir a maioria dentro de um *election timeout*.

3.3 etcd: arquitetura e modelo de dados

O etcd é um armazenamento chave-valor replicado pelo Raft, usado como banco de estado do Kubernetes. Cada membro guarda a base inteira; não há particionamento. Clientes falam com qualquer membro por gRPC; os membros falam entre si por HTTP na porta de pares (Figura 1, seção 4; caminho de uma escrita na Figura 2, E2). - **MVCC e revisões.** Toda alteração gera uma nova **revisão** global; cada chave guarda `create_revision`, `mod_revision` e `version`. A revisão ordena todas as escritas e serve de base para *watch* (notificações a partir

de uma revisão) e para transações condicionais. - **Leases.** Uma chave pode ser presa a um lease com prazo (TTL), renovado pelo cliente; se ele para de renovar, o lease vence e a chave é apagada. Travas (`lock`) e eleições (`elect`) de aplicação são construídas sobre leases e sobre a ordem das revisões. - **Leituras.** A leitura padrão é **linearizável**: o membro obtém do líder o índice já confirmado (*ReadIndex*), o líder confirma com um *heartbeat* que ainda é líder, e o membro responde depois de aplicar até esse índice, sem gravar nada no log. A leitura **serializável** responde com a cópia local, sem consultar ninguém. - **Snapshots e learners.** A cada `--snapshot-count` entradas o membro grava um snapshot e descarta o log antigo; um membro muito atrasado recebe o snapshot inteiro. *Learners* são membros que recebem o log sem votar, usados para adicionar um membro sem afetar o quórum enquanto ele se atualiza.

3.4 ZooKeeper e ZAB (HUNT et al., 2010; JUNQUEIRA; REED; SERAFINI, 2011)

O ZooKeeper organiza os dados numa árvore de **znodes** (caminhos como `/e9/trava`). Um znode **efêmero** existe enquanto a **sessão** do cliente que o criou existir; um znode **sequencial** recebe um número crescente no nome. Com esses dois tipos se constroem as receitas de trava e de eleição: cada cliente cria um znode efêmero e sequencial, e o de menor número é o dono; cada um observa só o anterior, o que evita que todos acordem ao mesmo tempo.

As escritas passam pelo líder e são difundidas pelo **ZAB** (*ZooKeeper Atomic Broadcast*), em ordem total. Cada transação recebe um **zxid** de 64 bits: os 32 bits altos são a **época** (número do mandato do líder) e os 32 baixos um contador que recomeça a cada época. O ZAB opera em fases: descoberta e sincronização (o novo líder estabelece a época e alinha o histórico dos seguidores, por DIFF, SNAP ou TRUNC) e difusão (propostas confirmadas pela maioria). As **leituras** são servidas pela cópia local do servidor ao qual o cliente está ligado, e podem estar atrasadas; o cliente que precisa do valor mais recente chama `sync` antes de ler. O ZooKeeper garante escritas linearizáveis e ordem FIFO das operações de cada cliente.

3.5 Conceitos da disciplina usados nos experimentos

- **Quórum.** Com N réplicas, decisões exigem a maioria (⌊N/2⌋ + 1); duas maiorias sempre se cruzam, o que impede decisões conflitantes. Com N = 3, a maioria é 2 e o sistema tolera f = 1 falha.

- **CAP e PACELC.** Sob partição (P), um sistema escolhe entre consistência (C) e disponibilidade (A) (GILBERT; LYNCH, 2002). O PACELC acrescenta que, sem partição (E), a escolha é entre latência (L) e consistência (C) (ABADI, 2012).
- **Bully e Anel.** Algoritmos clássicos de eleição em que vence o processo de maior identificador (Bully) ou em que uma mensagem percorre um anel lógico. Não usam termos, e uma partição pode produzir dois líderes.
- **Relógios lógicos.** O relógio de Lamport (LAMPORT, 1978) atribui números crescentes a eventos de forma consistente com a relação "aconteceu antes"; termos, épocas e revisões são contadores desse tipo.
- **Multicast de ordem total.** Todas as réplicas entregam as mensagens na mesma ordem; é o que o log do Raft e a difusão do ZAB oferecem.
- **Estado global.** O algoritmo de Chandy-Lamport (CHANDY; LAMPORT, 1985) obtém um corte consistente com marcadores; numa máquina de estados replicada, o estado em qualquer índice confirmado já é consistente, e o snapshot é a cópia desse estado.

4. AMBIENTE E REPRODUTIBILIDADE

4.1 Hardware e software (coletado em 24/10/2026)

Item	Valor
CPU	Intel Core i7-1355U (13ª geração)
RAM do host	15,7 GB (8 GB disponíveis à VM do Docker/WSL 2, 12 CPUs)
Sistema operacional	Windows 11 Pro; Docker Desktop com WSL 2 (kernel 5.15.167.4-microsoft-standard-WSL2)
Docker / Compose	Docker Engine 29.7.2 / Compose v5.5.1
etcd	3.5.17 (imagem `quay.io/coreos/etcd:v3.5.17`, 85,4 MB)
ZooKeeper	3.9.6 (imagem `zookeeper:3.9.6`, 461 MB), OpenJDK 17.0.20.1
Cliente ZK	`python:3.12-alpine` (74,7 MB) + kazoo 2.10.0

Evidência: `docs/ambiente/evidencias/`.

Os experimentos rodaram em duas máquinas. A segunda também serviu de teste de reprodutibilidade (R1): o repositório foi clonado e subiu do zero seguindo só o README (`docs/ambiente/evidencias/2026-10-26_2029_validacao_samuel.txt`).

Item	Máquina 1 (David): E0–E4, E9-E3/E4	Máquina 2 (Samuel): R1, E5–E8, E9-E5..E8
CPU	Intel Core i7-1355U	AMD Ryzen 5 5500
RAM do host	15,7 GB	15,9 GB
Sistema operacional	Windows 11 Pro	Windows 11 Home (10.0.26200)
VM do Docker (WSL 2)	8 GB, 12 CPUs	8 GB, 12 CPUs (kernel 5.15.146.1)
Docker Engine / Compose	29.7.2 / 5.5.1	29.9.2 / 5.6.1

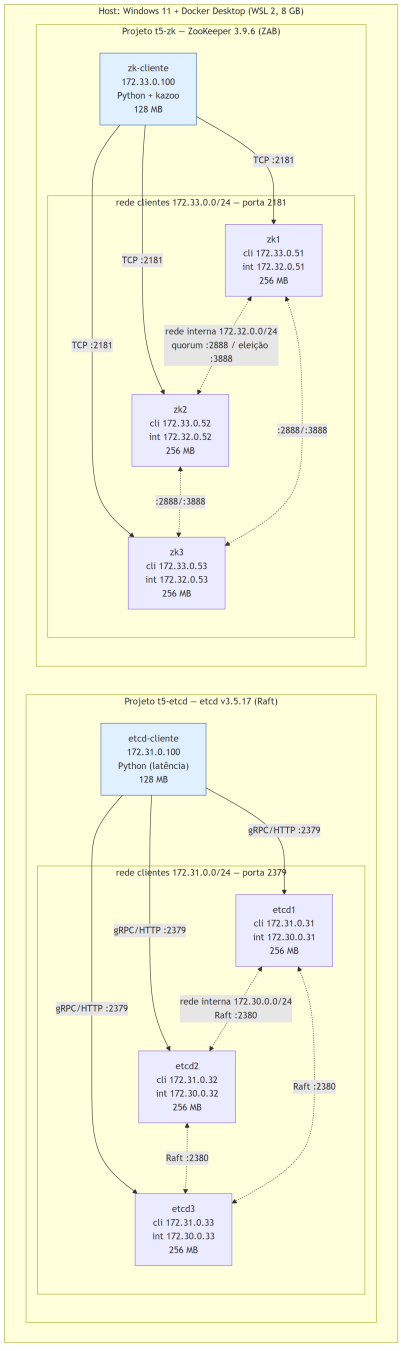
4.2 Topologia

Duas redes por laboratório: **interna** (só tráfego de consenso) e **clientes**. Isolar um nó da rede interna simula a partição sem perder o acesso do cliente a ele.

Nó	Rede interna	Rede clientes	Portas	mem_limit
etcd1	172.30.0.31	172.31.0.31	2380 pares / 2379 clientes	256 MB
etcd2	172.30.0.32	172.31.0.32	2380 / 2379	256 MB
etcd3	172.30.0.33	172.31.0.33	2380 / 2379	256 MB
zk1	172.32.0.51	172.33.0.51	2888 quorum, 3888 eleição / 2181	256 MB (heap 128 MB)
zk2	172.32.0.52	172.33.0.52	2888, 3888 / 2181	256 MB
zk3	172.32.0.53	172.33.0.53	2888, 3888 / 2181	256 MB
etcd-cliente	—	172.31.0.100	—	128 MB
zk-cliente	—	172.33.0.100	—	128 MB

Método de medição de latência. Os tempos de `put`/`get` (etcd) e `set`/`get` (ZooKeeper) são medidos por um cliente Python dentro da rede de clientes (`etcd-cliente` e `zk-cliente`), com `time.perf_counter`, uma conexão persistente com **um único** membro e a 1ª operação descartada como aquecimento. - No etcd, o cliente usa a API HTTP/JSON (gRPC-gateway). - No ZooKeeper, usa o kazoo 2.10.0.

Cronometrar no host com `docker exec etcdctl` foi descartado. Cada chamada cria um processo e uma conexão nova, e o overhead medido (cerca de 400 ms) é aproximadamente 100 vezes maior que a própria operação (achado A3, `docs/ambiente/evidencias/2026-10-25_1035_teste_ferramenta_latencia.txt`).



Fonte: `diagramas/topologia.mmd` (Mermaid); para gerar o PNG: `diagramas/renderizar.sh`. O caminho de uma escrita está em `diagramas/sequencia-escrita.png` (E2).

4.3 Memória medida

Situação	etcd (por membro)	ZooKeeper (por servidor)	Evidência
Repouso (24/10/2026)	23,7–29,7 MB	46,8–55,7 MB	`docs/ambiente/evidencias/2026-10-24_1614_memoria_repouso.txt`

Situação	etcd (por membro)	ZooKeeper (por servidor)	Evidência
Sob carga (logo após `check perf --load=s`, 25/10)	80,2–86,9 MiB por membro	82,7–132,8 MiB por servidor (logo após o E9-E3)	`experimentos/E3-replicacao/evidencias/2026-10-25_1309_memoria_apos_carga.txt`; `experimentos/E9-zookeeper/E3-replicacao/README.md`

Todos os valores ficaram abaixo do `mem_limit` de 256 MB, e o total dos dois laboratórios (6 nós + 2 clientes) fica abaixo de 1 GB.

4.4 Como reproduzir

Ver `README.md` na raiz (subida, validação, parada). Repositório: https://github.com/DavidSmithSouzak10/sd-t2-etcd-zookeeper (privado; acesso ao professor a combinar).

5. EXPERIMENTOS

Cada experimento seguiu o mesmo ciclo: validação do cluster (`scripts/validar.sh`), previsão da dupla registrada em commit **antes** da execução, execução por script (`experimentos/EX/executar.sh`), evidência bruta salva com data e hora e análise. As medições de tempo têm no mínimo 5 repetições (R7). Os detalhes completos, com a tabela previsão × resultado de cada um, estão no `README.md` de cada pasta em `experimentos/`.

Placar das previsões: E1–E4 e E9-E3/E4 (David), 14 confirmadas, 5 erradas e 1 não observada; E5–E8 e E9-E5..E8 (Samuel), 12 confirmadas, 1 errada e 1 parcial.

5.1 E1 — Subida e topologia

Objetivo: cluster com 3 membros no ar e papéis identificados. **Procedimento:** 5 subidas do zero (`down -v` + `up -d`), `member list`, `endpoint status` e `endpoint health`. **Resultado:** exatamente 1 líder em todas as subidas, escolhido ao acaso (etcd2, etcd1, etcd1, etcd3, etcd2); termo e RAFT INDEX iguais nos 3; saudável em 2,9 ± 0,8 s. A previsão de que o RAFT INDEX ficaria parado em repouso errou: ele sobe cerca de 0,6 entrada/s, porque o `endpoint health` do healthcheck do Compose é uma proposta pelo Raft. **Evidência:** `experimentos/E1-subida/evidencias/`; Figura 1 (topologia).

5.2 E2 — Caminho de uma operação

Objetivo: mostrar quem recebe, decide, replica e responde. **Procedimento:** `put` num seguidor por endpoint único, `-w json`, métricas de bytes entre pares e RAFT INDEX nos 3. **Resultado:** o seguidor **encaminha** ao líder (103 KB ao líder, 60 B ao outro seguidor), e o líder replica aos dois; a escrita é confirmada com 2 de 3 membros. O put via seguidor custou 8,25 ± 2,83 ms contra 6,91 ± 2,36 ms direto no líder (20 repetições). Revisão e RAFT INDEX são contadores diferentes (revisão 58 × índice 248). Achado: com só o líder vivo (`docker pause` nos outros dois) o cliente recebeu timeout, mas a escrita acabou confirmada ao despausar, ou seja, para o cliente um timeout é um resultado indeterminado. **Evidência:** `experimentos/E2-caminho/evidencias/`; Figura 2 (`diagramas/sequencia-escrita.png`).

5.3 E3 — Replicação

Objetivo: escrever num membro, ler em outro e medir a propagação. **Procedimento:** 20 repetições de `put` no etcd1, `get` no etcd3 e *watch* no etcd2; latências; `check perf --load=s`. **Resultado:** 20 de 20 leituras no outro membro viram o valor novo. *Watch* em 6,4 ± 1,3 ms; put 3,28 ms > get linearizável 1,46 ms > get serializável 0,76 ms. `check perf` passou (151 escritas/s), com no máximo 87 MiB por membro. **Evidência:** `experimentos/E3-replicacao/evidencias/`; `resultados/2026-10-25_1308_e3_*.csv`.

5.4 E4 — Consistência

Objetivo: evidenciar o modelo de consistência. **Procedimento:** linearizável × serializável no mesmo seguidor (20×); RAFT INDEX antes e depois de 100 leituras; seguidor isolado da rede interna. **Resultado:** serializável em 0,65 ms contra 1,82 ms da linearizável. 100 leituras linearizáveis não alteram o RAFT INDEX além da linha de base: o ReadIndex não grava no log (a previsão de que gravaria errou). Isolado, o seguidor falha na linearizável (`context deadline exceeded`, 5 de 5) e devolve `ANTIGO` na serializável (5 de 5) enquanto a maioria já tinha `NOVO`. **Evidência:** `experimentos/E4-consistencia/evidencias/2026-10-25_1310_seguidor_isolado.txt`.

5.5 E5 — Falha e quórum

Objetivo: derrubar 1 e depois 2 membros. **Procedimento:** 5 quedas do líder com `docker kill` e 5 com `docker stop`; quadro com 3, 2 e 1 membros. **Resultado:** novo líder em **1,46 ± 0,18 s** após `kill`, dentro do *election timeout* sorteado (1–2 s), com termo +1. Com `stop`, a troca levou 9,0 ± 1,7 ms, porque o etcd transfere a liderança

antes de sair (não previsto). Com 2 membros tudo funciona; com 1, escrita e leitura linearizável falham e a serializável responde.

Membros ativos	Escrita	Leitura linearizável	Leitura serializável
3	OK	OK	OK
2	OK	OK	OK
1	falha (`context deadline exceeded`)	falha	responde

Evidência: `experimentos/E5-falha-quorum/evidencias/2026-10-26_2120_quadro_quorum.txt`; `resultados/2026-10-26_2118_e5_eleicao_kill.csv`.

5.6 E6 — Partição de rede

Objetivo: isolar o líder da rede interna, mantendo-o acessível ao cliente. **Procedimento:** 5 isolamentos com medição da eleição; linha do tempo dos dois lados. **Resultado:** o lado majoritário elegeu novo líder em 0,94 ± 0,24 s (limite inferior, ver seção 9) com termo maior. O líder isolado deixou o cargo (`no leader`) já na primeira rodada; recusou as 6 escritas e todas as leituras linearizáveis; a leitura serializável devolveu `ANTIGO` enquanto a maioria lia `NOVO`. Depois de religar, os 3 membros ficaram com `NOVO`, sem divergência. Conclusão: o etcd escolhe **consistência** (PC/EC). **Evidência:** `experimentos/E6-particao/evidencias/2026-10-26_2121_linha_do_tempo.txt`.

5.7 E7 — Recuperação

Objetivo: religar o membro e medir o tempo até o cluster voltar a ficar saudável. **Procedimento:** 5 ciclos isolar → 1000 escritas → religar; depois, `--snapshot-count=1000`, seguidor parado durante 10 000 escritas. **Resultado:** o membro voltou como seguidor e com termo maior em 5 de 5, mas o tempo até alcançar o índice foi de 6,3–6,7 s em 4 de 5 repetições (50 ms em uma; média 5,2 ± 2,9 s). A cópia das entradas é imediata; o atraso está em restabelecer a comunicação após religar (causa não isolada, seção 9). A previsão de menos de 1 s errou. No teste de snapshot, o líder enviou um *merged snapshot* de 701 kB e o seguidor o restaurou; `endpoint hashkv` igual nos 3. **Evidência:** `experimentos/E7-recuperacao/evidencias/2026-10-26_2124_recuperacao_por_snapshot.txt`; `resultados/2026-10-26_2122_e7_recuperacao.csv`.

5.8 E8 — Coordenação e ordem

Objetivo: mostrar o mecanismo de ordenação e resolver exclusão mútua e eleição com o etcd. **Procedimento:** escritas alternando chaves; `etcdctl lock` e `elect` com dois competidores em contêineres próprios; queda do detentor (5×); fencing token. **Resultado:** a revisão cresce numa sequência única para todas as chaves; o termo só muda com eleição. Troca do `lock` em 7,9 ± 1,5 s (lease de 10 s). Troca do `elect` em **57,2 ± 0,3 s**: o `elect` não tem `--ttl` e usa a sessão padrão de 60 s (a previsão de cerca de 10 s errou). No `elect` venceu sempre a menor `create_revision`, isto é, quem pediu primeiro, e não o maior ID como no Bully. Com pausa longa do detentor, a gravação com fencing token foi recusada e a sem fencing sobrescreveu o trabalho do novo detentor. **Evidência:** `experimentos/E8-coordenacao/evidencias/2026-10-26_2141_troca_detentor.txt`, `…_2148_fencing_token.txt`.

5.9 E9 — Ênfase D: ZooKeeper (E3–E8) e comparação ZAB × Raft

A bateria E3–E8 foi repetida num ensemble de 3 servidores ZooKeeper 3.9.6 com cliente kazoo.

Sub	Resultado principal	Evidência
E3	*watch* em 5,9 ± 0,8 ms (disparo único); `set` no seguidor 7,0 ms; leitura local 1,53 ms × `sync` + leitura 3,89 ms; nenhuma leitura antiga em 20	`E9-zookeeper/E3-replicacao/`
E4	servidor isolado ainda serve leitura local antiga por cerca de 3,6 s e depois para de atender qualquer cliente; o `sync` falha em vez de devolver valor antigo	`E9-zookeeper/E4-consistencia/`
E5	eleição em **0,38 ± 0,09 s** após `kill` (a conexão fechada avisa os seguidores); 0,70 s após `stop`; época +1 em 10 de 10; sem maioria, o servidor fica sem serviço	`E9-zookeeper/E5-falha-quorum/`
E6	eleição em **3,6 ± 0,3 s** na partição (`syncLimit × tickTime` = 4 s); o líder isolado faz `Shutting down`; o cliente com a lista dos 3 migra em 3 ms	`E9-zookeeper/E6-particao/`
E7	sincronização por DIFF em 6 de 6, em 2,5 ± 0,7 s (200 transações)	`E9-zookeeper/E7-recuperacao/`
E8	trava e eleição com znodes efêmeros e sequenciais trocam em 7,9 s (sessão de 10 s); a época faz o papel do termo	`E9-zookeeper/E8-coordenacao/`

Comparação ZAB × Raft. Os dois garantem um líder por mandato (época ou termo), escrita confirmada pela maioria e consistência sob partição. As diferenças medidas são de projeto: o etcd lê de forma linearizável por padrão e detecta falhas só por tempo, com eleição em 1–2 s com ou sem partição; o ZooKeeper lê da cópia local por padrão (mais rápido, com janela de valor antigo, corrigida com `sync`)

e elege em 0,4 s quando a conexão fecha, mas leva 3,6 s numa partição. O quadro completo está em `experimentos/E9-zookeeper/comparacao.md`.

6. QUESTÕES COMUNS (Q1–Q14)

Q1 — Que problema o sistema resolve? Quais transparências oferece?

Resposta: O etcd guarda pouca informação crítica (configuração, descoberta de serviços, travas, eleição de líder) de forma consistente e disponível mesmo com a falha de uma minoria das réplicas. Para isso replica um log de comandos por consenso (Raft), formando uma máquina de estados replicada. Transparências: **replicação** (o cliente vê um único armazenamento, mas há 3 cópias idênticas), **falha** (a queda de 1 membro é mascarada: o cluster elege outro líder e continua), **localização** (qualquer membro atende; com a lista de endpoints, o cliente troca de membro sozinho), **acesso** (a mesma API em qualquer membro) e **concorrência** (as escritas são serializadas pelo líder; transações e travas resolvem disputas). **Evidência:** E3 (20 de 20 leituras corretas em outro membro); E5 (escritas continuam com 2 de 3).

Q2 — Arquitetura: estilo, componentes, papéis (diagrama próprio).

Resposta: Cliente-servidor, em que o "servidor" é um grupo de 3 réplicas com um **líder** e dois **seguidores** (nenhum *learner*). O líder é o único que anexa entradas ao log; os seguidores replicam e encaminham ao líder as escritas que recebem. Há duas redes: a interna, só para o tráfego Raft entre pares (porta 2380), e a de clientes (porta 2379). O *watch* acrescenta um componente orientado a eventos (notificação por *push*). O ZooKeeper tem a mesma estrutura (líder, seguidores, rede de quórum 2888 e eleição 3888). **Evidência:** Figura 1 (`diagramas/topologia.png`); Figura 2 (`diagramas/sequencia-escrita.png`); E1 (`member list`, `endpoint status`); E2 (encaminhamento ao líder).

Q3 — Como um cliente localiza um dado ou serviço?

Resposta: Não há particionamento nem cálculo de posição (nada de *hash* ou DHT): cada membro guarda a base inteira, e o dado é localizado pela **chave** num espaço plano ordenado (buscas por prefixo e por intervalo). O cliente pode falar com qualquer membro, a partir de uma lista de endpoints configurada. Para o próprio uso como serviço de nomes, o etcd funciona como um **diretório central replicado**: aplicações registram endereços em chaves e outras os consultam ou observam com *watch*. No ZooKeeper a resolução é **hierárquica**, por caminho na

árvore de znodes. **Evidência:** E1 (RAFT INDEX igual nos 3 membros); E3 (valor escrito no etcd1 lido no etcd3).

Q4 — Como os nós se comunicam? Protocolo, síncrono/assíncrono, semântica de entrega.

Resposta: Cliente ↔ membro: gRPC sobre HTTP/2, com mensagens Protocol Buffers, requisição e resposta **síncronas**; o *watch* é um *stream* assíncrono em que o servidor empurra eventos. Entre pares: mensagens Raft (AppendEntries, votos, *heartbeats*) por HTTP (rafthttp), de forma **assíncrona**. Semântica de entrega: cada entrada confirmada é aplicada **exatamente uma vez**, na ordem do log, em todas as réplicas; para o cliente, porém, um timeout é **indeterminado**, porque a escrita pode ter sido confirmada mesmo sem resposta. Se o cliente repete, a semântica efetiva vira "ao menos uma vez", e operações não idempotentes precisam de cuidado (por exemplo, transação condicionada à revisão). O ZooKeeper usa um protocolo próprio sobre TCP, com ordem FIFO por sessão. **Evidência:** E2 (`2026-10-25_1305_metricas_encaminhamento.txt`; timeout com escrita confirmada em `2026-10-25_1306_logs_raft_pausa.txt`); E3 (*watch* em 6,4 ms).

Q5 — Modelo de replicação; quantas réplicas; quem aceita escritas.

Resposta: Máquina de estados replicada com **líder único**: 3 réplicas completas; só o líder aceita escritas (um seguidor que recebe uma escrita a encaminha ao líder), e cada escrita é confirmada quando a maioria (2 de 3) a grava. O ZooKeeper segue o mesmo modelo (líder + difusão atômica ZAB). **Evidência:** E2 (103 KB do seguidor para o líder, 60 B para o outro seguidor; escrita confirmada com 2 de 3); E9-E3.

Q6 — Modelo de consistência (demonstrar com experimento).

Resposta: etcd: **linearizável** por padrão, para escritas e leituras. A leitura padrão usa ReadIndex e, sem líder alcançável, falha em vez de devolver valor antigo. A leitura **serializável** (`--consistency=s`) é opcional e pode devolver valor antigo. ZooKeeper: escritas linearizáveis, mas leituras **locais** por padrão, com garantia de ordem por cliente (consistência sequencial); a leitura atual exige `sync`. **Evidência:** E3 (20 de 20 leituras com o valor novo); E4 (no seguidor isolado, linearizável `context deadline exceeded` 5 de 5 e serializável `ANTIGO` 5 de 5); E9-E4 (servidor isolado serve leitura local antiga por cerca de 3,6 s); E6 (lado majoritário lê `NOVO`, minoritário serializável lê `ANTIGO`).

Q7 — Quórum: N, R, W ou maioria; falhas toleradas; perder o quórum.

Resposta: Não há R e W configuráveis: toda escrita e toda leitura linearizável exigem a **maioria** (N = 3, maioria = 2). Com isso, duas maiorias sempre se cruzam (2 + 2 > 3, o equivalente a R + W > N), e o cluster tolera f = 1 falha (N = 2f + 1). Perdendo o quórum, escrita e leitura linearizável falham; só a leitura serializável responde. No ZooKeeper, sem quórum o servidor para de atender completamente. **Evidência:** E5 (`2026-10-26_2120_quadro_quorum.txt`); E2 (`2026-10-25_1305_confirmacao_maioria.txt`); E9-E5 (`2026-10-26_2149_quadro_quorum.txt`).

Q8 — Líder: como é escolhido; comparar com Bully e Anel.

Resposta: Pelo Raft: um seguidor que fica um *election timeout* (1–2 s, sorteado) sem *heartbeat* se candidata num termo novo; cada nó vota uma vez por termo e só em candidatos com log tão atualizado quanto o seu; vence quem tem a maioria. O resultado é imprevisível (qualquer membro pode vencer). No **Bully** vence sempre o maior identificador e no **Anel** uma mensagem de eleição percorre os processos; nenhum dos dois tem termos nem exige maioria, então uma partição pode gerar dois líderes. No ZooKeeper, a eleição (FastLeaderElection) prefere o servidor com o histórico mais novo (maior zxid) e, em empate, o maior ID. **Evidência:** E1 (líder diferente a cada subida); E5 (eleição em 1,46 ± 0,18 s, termo +1); E8 (`elect` vence a menor `create_revision`); E9-E5 (0,38 ± 0,09 s, época +1).

Q9 — Ordenação de eventos/versões; relação com Lamport, vetoriais, multicast ordenado.

Resposta: O etcd usa três contadores: o **termo** (época de liderança; só muda com eleição), o **índice** do log (posição de cada entrada no consenso) e a **revisão** (contador global do banco; sobe a cada escrita, em ordem total para todas as chaves). A revisão funciona como um relógio de Lamport único, mantido pelo líder: ordena tudo, mas não registra concorrência, como faria um relógio vetorial, porque o Raft serializa as escritas. O log replicado é um **multicast de ordem total**: todos aplicam as mesmas entradas na mesma ordem. No ZooKeeper o zxid junta época e contador num só número. **Evidência:** E8 (`2026-10-26_2141_relogios_logicos.txt`: revisões 58–63 alternando chaves); E2 (revisão 58 × índice 248); E9-E8 (`2026-10-26_2139_epoca_x_termo.txt`).

Q10 — Modelo de falhas; detecção; falha de 1 nó × da maioria.

Resposta: Falhas por parada (*crash*) com recuperação; mensagens podem se perder ou atrasar; não há tolerância a falhas bizantinas. Detecção por *heartbeat* e timeout: o líder envia *heartbeats* a cada 100 ms; sem eles por um *election timeout*, os seguidores elegem outro; com *check quorum*, o líder que não ouve a maioria

deixa o cargo. Falha de 1 nó: mascarada (nova eleição em 1–2 s, se era o líder). Falha da maioria: o cluster para de aceitar escritas e leituras linearizáveis. **Evidência:** E5 (eleição após `kill`; quadro com 1 membro: `etcdserver: no leader`); E6 (líder isolado sem líder já na 1ª rodada); E9-E6 (detecção do ZooKeeper em cerca de 4 s, `syncLimit × tickTime`).

Q11 — Partição: consistência ou disponibilidade? (CAP, PACELC; sustentar com E6).

Resposta: Consistência. No E6, o lado majoritário elegeu outro líder e continuou atendendo; o lado minoritário (o antigo líder) recusou todas as escritas e leituras linearizáveis, e nenhuma versão divergente surgiu. A única resposta do lado minoritário foi a leitura serializável, com o valor antigo, que o cliente precisa pedir explicitamente. Pelo PACELC, o etcd é PC/EC: sem partição, também paga latência pela consistência (leitura linearizável 1,82 ms × serializável 0,65 ms). O ZooKeeper também é CP, e ainda mais restritivo: o isolado para de responder até leituras. **Evidência:** E6 (`2026-10-26_2121_linha_do_tempo.txt`); E4 (latências); E9-E6 (`2026-10-26_2132_linha_do_tempo_so_lider.txt`).

Q12 — Recuperação de estado: log, snapshot; relação com estado global.

Resposta: Incremental quando o líder ainda tem as entradas que faltam: ele envia só essas entradas. **Completa** (snapshot) quando o log já foi compactado: o líder envia o estado inteiro. No ZooKeeper o equivalente é DIFF (incremental), SNAP (completo) ou TRUNC (descartar transações não confirmadas). Relação com estado global: numa máquina de estados replicada, o estado em qualquer índice confirmado já é um corte consistente, porque todos aplicam a mesma sequência; por isso o snapshot não precisa de marcadores como no algoritmo de Chandy-Lamport. **Evidência:** E7 (1005 entradas recuperadas pelo log; snapshot de 701 kB, `restored snapshot`; `endpoint hashkv` igual nos 3); E9-E7 (DIFF em 6 de 6).

Q13 — Problemas clássicos presentes e como são resolvidos/evitados.

Resposta: Consenso e **multicast de ordem total**: resolvidos pelo Raft (log replicado) e pelo ZAB. **Eleição de líder**: interna (Raft, com termos e maioria) e oferecida às aplicações (`etcdctl elect`; receita de eleição do ZooKeeper). **Exclusão mútua**: `etcdctl lock` com lease, e receita de trava com znodes efêmeros e sequenciais; a trava se libera sozinha quando o detentor morre (lease ou sessão expira). **Generais bizantinos**: não tratado, porque o modelo assume falhas por parada. Limite conhecido: trava com lease não protege contra detentor pausado; é preciso fencing token (S4). **Evidência:** E8 (troca do `lock` em 7,9 s; fencing); E9-E8 (troca da trava em 7,9 s).

Q14 — Uso na prática, limitações, cuidados de segurança.

Resposta: Uso: o etcd é o banco de estado do Kubernetes (todos os objetos do cluster) e é usado para descoberta de serviços e configuração; o ZooKeeper coordena sistemas como HBase, Hadoop e, nas versões anteriores ao KRaft, o Kafka. **Limitações:** guarda pouco dado (requisição de até 1,5 MiB no etcd e cerca de 1 MB no ZooKeeper); a vazão de escrita é limitada por um único líder e pelo fsync; a latência cresce com a distância entre membros; sem maioria, o sistema para. **Segurança:** usar TLS mútuo entre membros e com clientes, autenticação e controle de acesso (RBAC no etcd, ACLs no ZooKeeper), e restringir a rede de pares. O laboratório usa texto plano sem autenticação (seção 9). **Evidência:** E0 (1,4 MiB aceito e 1,6 MiB recusado no etcd com `request is too large`; ZooKeeper aceita 900 KB e derruba a conexão com 1,1 MB); E3 (`check perf`: 151 escritas/s).

7. QUESTÕES ESPECÍFICAS DO TEMA 5 (S1–S4)

S1 — O que o Raft acrescenta a Bully e Anel para a eleição ser segura (termos, voto único por termo, restrição de log)? Por que o timeout é aleatório?

Resposta: Bully e Anel assumem detecção de falhas confiável e não têm como distinguir um líder antigo de um novo; numa partição, cada lado pode eleger o seu. O Raft acrescenta três mecanismos (ONGARO; OUSTERHOUT, 2014, §5.2 e §5.4): - **termos**: cada eleição abre um termo maior, e qualquer mensagem de termo menor é recusada, então um líder antigo que volta é rebaixado; - **voto único por termo** com **maioria**: como duas maiorias se cruzam, no máximo um candidato vence em cada termo; - **restrição de log**: só recebe voto quem tem log tão atualizado quanto o do eleitor, e por isso o vencedor já contém todas as entradas confirmadas.

O timeout é **aleatório** para que os seguidores raramente expirem juntos: se vários se candidatassem ao mesmo tempo, dividiriam os votos e ninguém teria maioria. O sorteio faz um deles quase sempre largar na frente. Por isso o líder não depende do identificador, ao contrário do Bully. O etcd 3.5 acrescenta o *pre-vote*, que impede um nó isolado de inflar o termo. **Evidência:** E1 (líder diferente a cada subida); E5 (eleição em 1,46 ± 0,18 s, dentro do intervalo sorteado de 1–2 s; termo +1); E6 (o isolado ficou no termo 18 enquanto a maioria foi para 19, e não forçou eleição ao voltar); E2 (logs de *check quorum* e *pre-vote*).

S2 — Por que 3 ou 5 membros e não 4? Custo de latência de aumentar membros? Para que servem os learners?

Resposta: A tolerância depende da maioria: com N membros, a maioria é ⌊N/2⌋ + 1 e o cluster tolera N − maioria falhas. Com 3, tolera 1; com 4, a maioria passa a 3 e continua tolerando só 1; com 5, tolera 2. Um quarto membro não acrescenta tolerância, só acrescenta mais um voto a esperar e mais uma réplica a manter. **Latência:** cada escrita só é confirmada quando a maioria responde. Mais membros significam uma maioria maior (o commit espera o k-ésimo mais rápido), mais mensagens e mais banda saindo do líder, então a escrita fica mais lenta e a vazão cai. **Learners** são membros que recebem o log sem votar. Servem para adicionar um membro novo: ele se atualiza como learner, sem entrar na conta da maioria, e só é promovido a votante quando alcança o líder. Assim a entrada de um membro atrasado não reduz a disponibilidade. **Evidência:** E5 (com 2 de 3 tudo funciona; com 1 de 3, nada que exija maioria); E2 (escrita confirmada com 2 de 3); E1 (`member list` sem learners). Os learners não foram exercitados (ênfase C).

S3 — Leitura linearizável × serializável; como o etcd implementa a primeira sem gravar no log? Onde o Kubernetes usa cada uma?

Resposta: A **linearizável** devolve sempre o valor da última escrita confirmada antes do início da leitura; a **serializável** devolve a cópia local do membro, que pode estar atrasada. O etcd implementa a linearizável com **ReadIndex** (ONGARO; OUSTERHOUT, 2014, §8): o membro pede ao líder o índice confirmado atual; o líder confirma, com uma rodada de *heartbeat* para a maioria, que ainda é o líder; o membro espera ter aplicado até esse índice e responde. Nada é gravado no log. O custo é uma ida ao líder mais uma rodada de *heartbeat*. **Kubernetes:** quando o cliente pede o estado mais recente (`resourceVersion` vazio), o kube-apiserver precisa devolver um dado consistente: tradicionalmente, uma leitura de quórum (linearizável) no etcd; desde a versão 1.31, ele pode servir essa leitura do seu cache de *watch*, que mantém em dia com o etcd por meio de notificações de progresso. Quando o cliente aceita qualquer versão (`resourceVersion="0"`), a resposta vem sempre do cache, sem garantia de atualidade. **Evidência:** E4 (100 leituras linearizáveis não alteram o RAFT INDEX além da linha de base; linearizável 1,82 ms × serializável 0,65 ms; no isolado, linearizável falha e serializável devolve `ANTIGO`); E9-E3 e E9-E4 (no ZooKeeper, o equivalente é `sync` + leitura: 3,25 ms × 1,03 ms).

S4 — Travas com lease e pausa longa do detentor; como um fencing token (revisão da chave) resolve.

Resposta: A trava com lease se libera sozinha quando o detentor deixa de renovar. Se o detentor sofre uma pausa longa (coleta de lixo, máquina congelada, rede lenta) maior que o TTL, o lease vence e outro cliente pega a trava. Quando o primeiro acorda, ele ainda acredita ser o dono e pode gravar no recurso protegido, desfazendo o trabalho do novo dono (KLEPPMANN, 2016). O **fencing token** é um número que cresce a cada nova aquisição da trava; no etcd, a revisão de criação da chave da trava. O detentor envia o token junto com cada gravação, e o recurso (ou uma transação condicionada) recusa tokens menores que o maior já visto. Assim o detentor antigo, com token menor, é barrado. **Evidência:** E8 (`2026-10-26_2148_fencing_token.txt`: A pegou a trava com token 113 e TTL 5 s, pausou 9 s; B pegou com token 114; a gravação de A **com** fencing foi recusada e **sem** fencing sobrescreveu o trabalho de B). No ZooKeeper, o `czxid` do znode da trava cresce a cada aquisição e pode cumprir o mesmo papel (E9-E8).

8. QUADRO-SÍNTESE

Coluna ZooKeeper acrescentada por causa da Ênfase D. Toda célula referencia a evidência.

Aspecto	etcd	ZooKeeper	Evidência
Estilo arquitetural e papéis	cliente-servidor; grupo de 3 réplicas com 1 líder e 2 seguidores; *watch* orientado a eventos	idem (leader, 2 followers)	E1; E2; Figura 1
Como o dado ou serviço é localizado	por chave, num espaço plano; cada membro tem a base inteira; qualquer membro atende	por caminho na árvore de znodes; cada servidor tem a árvore inteira	E1; E3; E9-E3
Modelo de replicação (N réplicas; quem aceita escritas)	máquina de estados replicada; N = 3; só o líder escreve, seguidores encaminham	idem, por difusão atômica (ZAB)	E2; E9-E3
Modelo de consistência observado	linearizável por padrão (ReadIndex); serializável opcional, pode devolver valor antigo	escritas linearizáveis; leitura local por padrão (pode ser antiga); `sync` para leitura atual	E4; E9-E4
Quórum (N, R, W ou maioria) e falhas toleradas	maioria 2 de 3; tolera 1 falha; sem maioria só a leitura serializável responde	maioria 2 de 3; tolera 1; sem maioria não atende ninguém	E5; E9-E5

Aspecto	etcd	ZooKeeper	Evidência
Líder ou coordenador e forma de escolha	Raft: timeout sorteado (1–2 s), voto único por termo; 1,46 ± 0,18 s	FastLeaderElection (maior zxid, depois maior ID); 0,38 ± 0,09 s após queda abrupta	E5; E9-E5
Mecanismo de ordenação ou versionamento	termo (mandato), índice do log, revisão global (ordem total)	zxid = época + contador	E8; E2; E9-E8
Comportamento sob partição (C ou A)	C: minoria sem líder recusa escrita e leitura linearizável; maioria elege em 0,94 s (PC/EC)	C: minoria para de atender após cerca de 4 s; maioria elege em 3,6 s	E6; E9-E6
Mecanismo de recuperação	entradas faltantes pelo log; snapshot se o log foi compactado; estado final idêntico (hash)	DIFF, SNAP ou TRUNC; DIFF observado em 6 de 6	E7; E9-E7
Problemas clássicos presentes e como são tratados	consenso e ordem total (Raft); eleição e exclusão mútua (`elect`, `lock` com lease); fencing token pela revisão; sem tolerância bizantina	consenso e ordem total (ZAB); trava e eleição com znodes efêmeros e sequenciais e sessão	E8; E9-E8

9. LIMITAÇÕES E AMEAÇAS À VALIDADE

O que um laboratório num único computador não mostra - **Rede.** Todos os nós estão no mesmo host: não há latência, perda nem largura de banda reais entre membros. Os tempos de replicação (milissegundos) seriam maiores entre máquinas e muito maiores entre regiões, e a escrita seria mais afetada que a leitura serializável. - **Disco e CPU compartilhados.** Os 6 nós disputam o mesmo SSD e a mesma CPU; uma falha de disco ou um disco lento de um único membro não pode ser simulado. - **Relógio compartilhado.** Os contêineres usam o relógio do host, então desvios de relógio não aparecem. O Raft não depende de relógio para a segurança, mas leases e sessões dependem de medir tempo; num cluster real, desvio ou pausa do relógio afeta o momento em que uma trava se libera (S4). - **Falhas simuladas.** `docker kill`, `docker stop`, `docker pause` e `docker network disconnect` imitam queda, parada ordenada, congelamento e partição, mas não falhas parciais (perda intermitente de pacotes, partição assimétrica).

Ameaças à validade das medições - **Duas máquinas.** E1–E4 e E9-E3/E4 rodaram na máquina 1 (Intel i7, Docker Engine 29.7.2); E5–E8 e E9-E5..E8 na máquina 2 (AMD Ryzen 5, Docker Engine 29.9.2) (seção 4). Comparações de tempo entre experimentos de máquinas diferentes devem ser lidas com cuidado. -

Amostras pequenas. A maioria das medições tem n = 5 (o mínimo do R7) ou n = 20. Algumas observações têm uma única repetição e estão marcadas como tal: snapshot (E7), 5000 transações (E9-E7) e trocas com `docker stop` (E8). - **Início dos eventos de rede.** Três das 5 eleições do E6 ficaram abaixo de 1 s, menos que o *election timeout* mínimo. Isso indica que o evento `disconnect` do Docker é registrado depois do corte real (o mínimo teórico é cerca de 0,9 s: o *election timeout* de 1 s menos um *heartbeat*). Os tempos medidos a partir de eventos de rede (`disconnect` no E6 e no E9-E6, `connect` no E7 e no E9-E7) podem estar subestimados em até cerca de 0,3–0,4 s; os tempos medidos a partir de `kill` (E5, E9-E5, E8) não são afetados. - **Atraso não explicado no E7.** O membro religado levou cerca de 6,5 s para voltar a ouvir o líder em 4 de 5 repetições (50 ms em uma). A causa não foi isolada; as hipóteses são o intervalo de nova tentativa de conexão entre pares do etcd e a resolução do endereço do contêiner religado na rede do Docker. O E4 observou o mesmo fenômeno (7 s). - **Leitura de estado durante a transição.** No E5, uma repetição registrou termo 3 → 3 porque o termo "depois" foi lido de um membro ainda não atualizado; o tempo medido não é afetado. - **Overhead do Docker.** Comandos via `docker exec` custam de 250 a 400 ms (256,6 ± 37,2 ms em 24/10; 407,2 ± 35,7 ms em 25/10). Mitigação: latências medidas por um cliente dentro da rede de clientes (seção 4); eventos medidos com o horário do próprio Docker (`docker events`). - **Atividade de fundo.** O healthcheck do Compose grava no log do Raft (cerca de 0,6 entrada/s), então o RAFT INDEX nunca fica parado; as comparações de índice usam linha de base (E1, E4). - **Execuções descartadas.** Duas execuções com defeito de script foram descartadas e refeitas (E8: `elect` sem `--ttl`; E9-E5: znode pai inexistente). O motivo está no diário (`relatorio/notas/diario.md`, sessão 9). - **Fuso horário.** Os registros dentro dos contêineres estão em UTC; o host está em UTC−3.

Segurança - Comunicação em texto plano, sem TLS, sem autenticação e sem controle de acesso, o que é aceitável só em laboratório (Q14).

10. PAPEL DE CADA INTEGRANTE

Integrante	Pesquisa	Experimentos	Escrita
David	Raft, replicação e commit, MVCC, *watch*, leases, ReadIndex, leitura local × `sync`	ambiente (Compose, scripts de validação e medição), diagramas; previsões e execução de E0–E4 e E9-E3/E4; scripts de E5–E8 e E9-E5..E8	seções 1 e 4; análises de E1–E4 e E9-E3/E4

Integrante	Pesquisa	Experimentos	Escrita
Samuel	eleição, *pre-vote* e *check quorum*, snapshots, ZAB e zxid, CAP/PACELC, Bully e Anel, relógios lógicos, Chandy-Lamport	teste de reprodutibilidade do zero (R1); previsões de E5–E8 e E9-E5..E8; execução de E5–E8 e E9-E5..E8 na máquina 2	análises de E5–E8 e E9-E5..E8; quadro comparativo do E9

As seções 2, 3, 5 a 9 e 11 foram revisadas e aprovadas pela dupla em 28/10/2026. Os dois apresentam o seminário e respondem por todas as questões (R10). Fonte: `docs/equipe.md`, `CHECKLIST.md` e histórico do repositório.

11. DECLARAÇÃO DE USO DE FERRAMENTAS DE IA GENERATIVA

Usamos o **Claude** (Anthropic), pela interface web e pelo **Claude Code** no VS Code, ao longo de todo o trabalho. O registro completo, com data, parte e forma de verificação, está em `docs/uso-ia.md`. Em resumo:

Parte	Uso da IA	Como verificamos
Ambiente	escreveu os arquivos Compose, os scripts de validação, medição e partição e os clientes de medição	clusters subidos e validados em duas máquinas diferentes; teste de reprodutibilidade do zero (R1)
Previsões	apresentou perguntas e conceitos e redigiu o texto a partir das escolhas da dupla; nas previsões de E5–E9, a pedido da dupla, conferiu a coerência com a teoria e reescreveu no formato do roteiro, **antes de qualquer execução**	cada previsão foi registrada em commit anterior à execução; a dupla leu e aprovou o texto
Execução	rodou os scripts, diagnosticou e corrigiu defeitos (execuções com defeito descartadas e registradas)	evidências brutas geradas pelos scripts, com data e hora, sem edição manual
Análise e relatório	redigiu rascunhos das análises, das respostas Q1–Q14 e S1–S4, dos quadros e das seções 2, 3, 5, 8 e 9	todo número conferido contra os CSVs (`scripts/estatistica.py`); toda afirmação aponta para uma evidência ou fonte primária; causas não verificadas marcadas como hipótese; texto revisado e aprovado pela dupla em 28/10/2026

A IA não escolheu as previsões no lugar da dupla, não inventou resultados e não editou evidências. A responsabilidade pelo conteúdo e pela defesa na arguição é dos autores.

12. REFERÊNCIAS (ABNT)

ONGARO, D.; OUSTERHOUT, J. In search of an understandable consensus algorithm. In: USENIX ANNUAL TECHNICAL CONFERENCE (USENIX ATC), 2014, Philadelphia. **Proceedings** [...]. Berkeley: USENIX Association, 2014. p. 305-319.

HUNT, P. et al. ZooKeeper: wait-free coordination for internet-scale systems. In: USENIX ANNUAL TECHNICAL CONFERENCE (USENIX ATC), 2010, Boston. **Proceedings** [...]. Berkeley: USENIX Association, 2010.

JUNQUEIRA, F. P.; REED, B. C.; SERAFINI, M. Zab: high-performance broadcast for primary-backup systems. In: IEEE/IFIP INTERNATIONAL CONFERENCE ON DEPENDABLE SYSTEMS AND NETWORKS (DSN), 41., 2011, Hong Kong. **Proceedings** [...]. IEEE, 2011. p. 245-256.

BURROWS, M. The Chubby lock service for loosely-coupled distributed systems. In: SYMPOSIUM ON OPERATING SYSTEMS DESIGN AND IMPLEMENTATION (OSDI), 7., 2006, Seattle. **Proceedings** [...]. Berkeley: USENIX Association, 2006.

KLEPPMANN, M. **How to do distributed locking**. 2016. Disponível em: https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html. Acesso em: 28 out. 2026.

ETCD AUTHORS. **etcd documentation v3.5**. Disponível em: https://etcd.io/docs/v3.5/. Acesso em: 28 out. 2026.

THE APACHE SOFTWARE FOUNDATION. **Apache ZooKeeper 3.9 documentation**. Disponível em: https://zookeeper.apache.org/doc/current/index.html. Acesso em: 28 out. 2026.

RAFT. **The Raft consensus algorithm**. Disponível em: https://raft.github.io. Acesso em: 28 out. 2026.

ABADI, D. Consistency tradeoffs in modern distributed database system design: CAP is only part of the story. **Computer**, v. 45, n. 2, p. 37-42, fev. 2012.

CHANDY, K. M.; LAMPORT, L. Distributed snapshots: determining global states of distributed systems. **ACM Transactions on Computer Systems**, v. 3, n. 1, p. 63-75, fev. 1985.

GILBERT, S.; LYNCH, N. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. **ACM SIGACT News**, v. 33, n. 2, p. 51-59, jun. 2002.

LAMPORT, L. Time, clocks, and the ordering of events in a distributed system. **Communications of the ACM**, v. 21, n. 7, p. 558-565, jul. 1978.

KUBERNETES AUTHORS. **Kubernetes API concepts**: resource versions. Disponível em: https://kubernetes.io/docs/reference/using-api/api-concepts/. Acesso em: 28 out. 2026.